Guía de sustentación — Portal de pagos

Oct 15, 2026 · @David Lopez

En la sesión de 60 minutos tienes que explicar el diseño y el código, y hacer un cambio en vivo siguiendo SDD: spec → test → código. Aquí tienes el orden de la sesión, seis diagramas para explicar el sistema, dónde está cada pieza (archivo:línea), respuestas cortas a las preguntas probables y cambios ya ensayados.

Agenda de los 60 minutos

Lo más importante es el cambio en vivo hecho con SDD; todo lo demás lo prepara. Si los evaluadores llevan el orden, usa cada bloque como módulo suelto.

Minutos	Bloque	Qué muestras
0–2	Pitch	El sistema en seis frases (sección siguiente)
2–7	SDD	`CLAUDE.md` → un criterio de `docs/spec.md` → `docs/openapi.yaml` → `docs/traceability.md` (59/59)
7–12	Arquitectura	Diagrama, `docker-compose.yml` y un rol de PostgreSQL por esquema
12–22	Core de pagos	Máquina de estados → transferencia → idempotencia → riel y worker
22–27	Eventos	Outbox en la misma transacción → relay → consumidor de Audit idempotente
27–32	Demo	Transferencia interna y externa, replay de idempotencia, timeline de auditoría
32–50	Cambio en vivo	Criterio en la spec → test en rojo → código → verde → `trace.mjs` → commit pequeño
50–60	Preguntas	Sección de preguntas probables

Regla para toda la sesión: ante cada pregunta, abre el archivo y señala la línea en vez de responder de memoria.

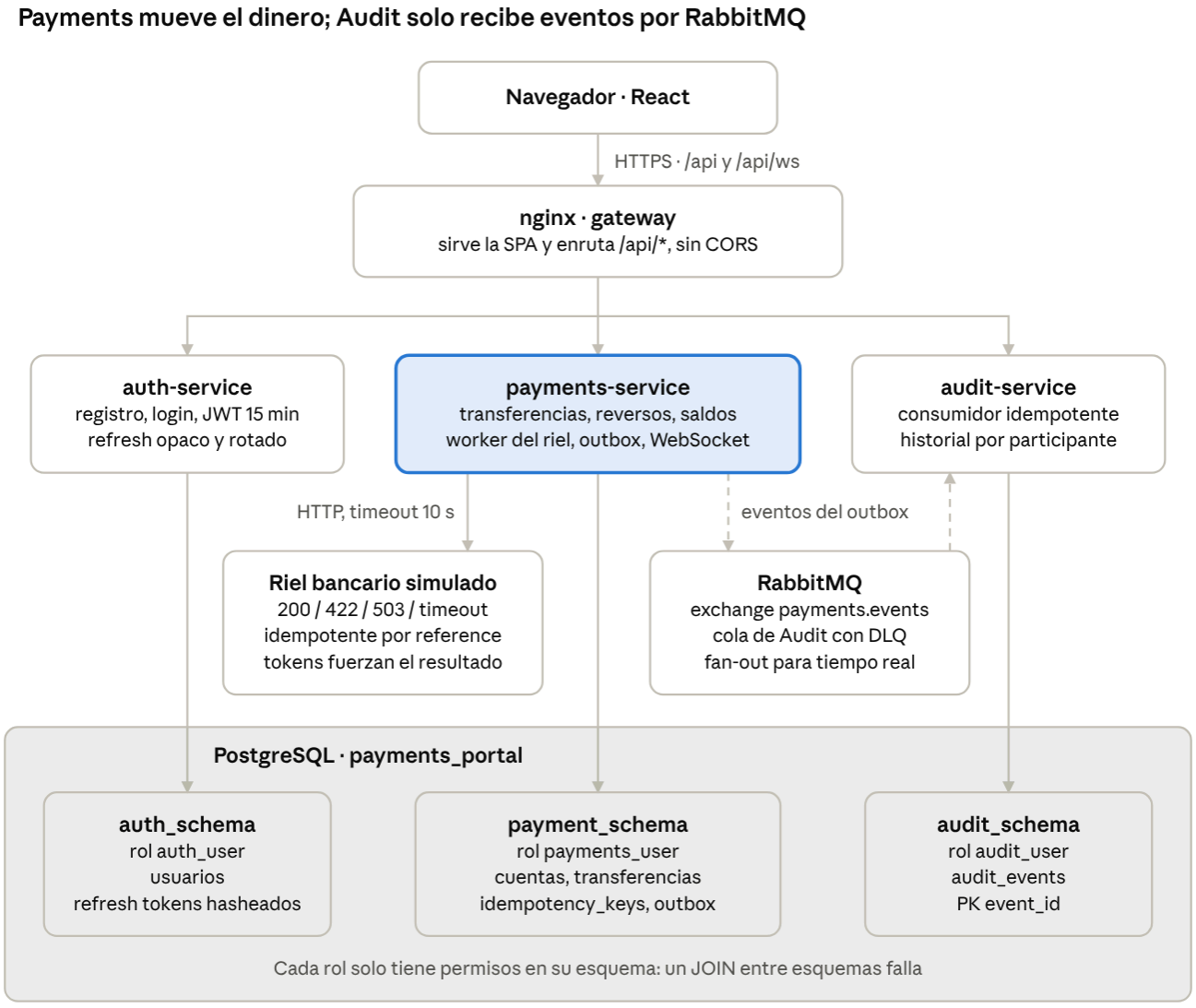
Pitch de apertura (2 minutos)

Seis ideas en este orden; dílas con tus palabras, no las leas.

1. Construí un mini portal de pagos con Spec-Driven Development. Antes del código fijé 59 criterios Given/When/Then, el contrato OpenAPI y las reglas del agente en `CLAUDE.md`; ese commit, `spec: initial`, quedó a las 08:58.
2. Son tres microservicios hexagonales en Node 22 y TypeScript (Auth, Payments y Audit), un simulador del riel bancario y un frontend en React. Cada servicio tiene su esquema y su propio rol de PostgreSQL, sin consultas cruzadas.
3. El corazón es Payments. El dinero vive en centavos como `bigint` y cada movimiento ocurre en una transacción que bloquea las cuentas en orden de id. La llave de idempotencia se guarda en esa misma transacción: diez peticiones iguales a la vez mueven el dinero una sola vez.
4. Una transferencia externa queda `PENDING` con el dinero retenido. Un worker la envía al riel con el id de la transferencia como referencia idempotente. Un timeout nunca es `FAILED`: es `UNKNOWN`, y antes de reenviar se le pregunta al banco.
5. Cada cambio de estado escribe un evento en un outbox dentro de la misma transacción. Un relay lo publica en RabbitMQ con confirmaciones y Audit lo consume de forma idempotente; si el broker se cae, no se pierde nada.
6. Hay 196 tests nombrados con el ID del criterio que cubren, y un script falla si algún criterio queda sin test. Todo levanta con `docker compose up` y está desplegado en AWS.

Cierra pasando el control: «¿Prefieren que empiece por la spec o por el código?»

Arquitectura de un vistazo



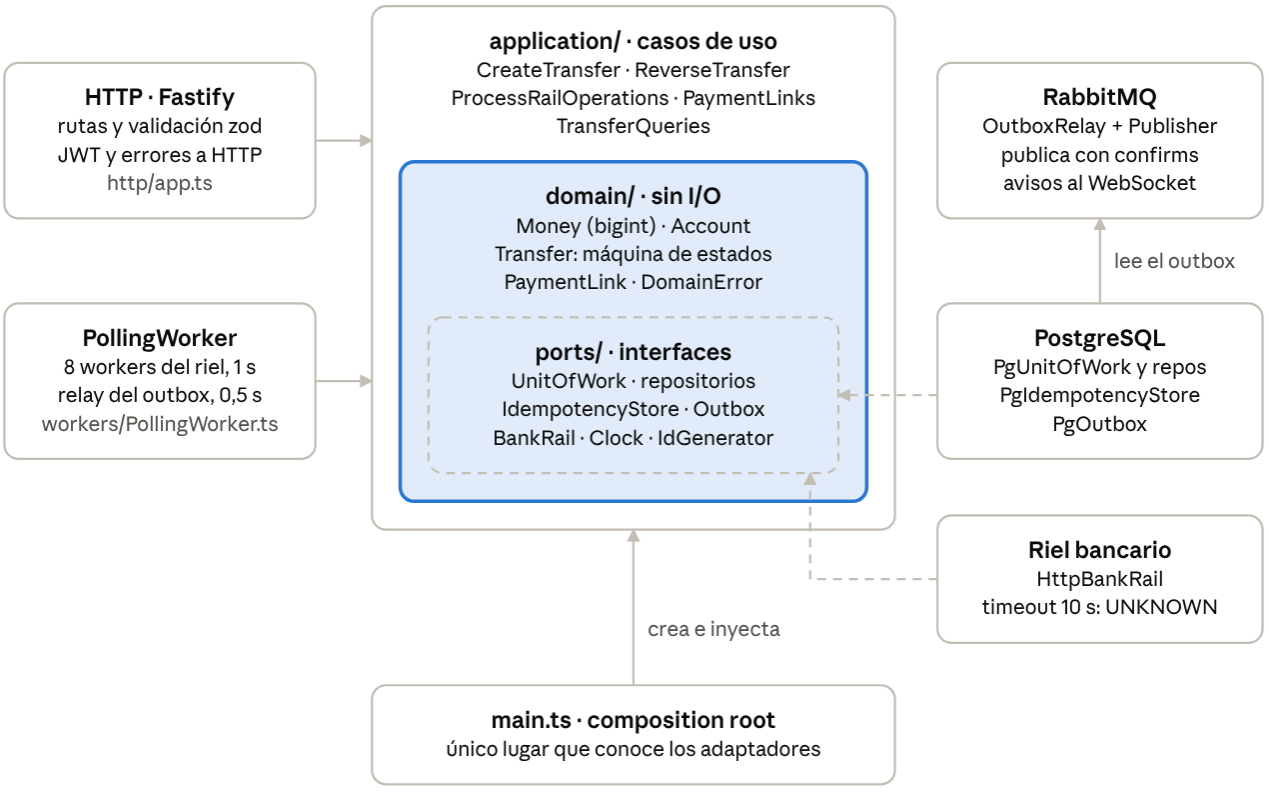
Línea continua: llamada síncrona · discontinua: evento asíncrono por RabbitMQ

arquitectura · 3 servicios, riel, RabbitMQ y PostgreSQL

Payments es el único que mueve dinero y habla con el riel. Audit se entera solo por eventos de RabbitMQ, y cada servicio entra a PostgreSQL con un rol que solo ve su esquema.

Hexagonal dentro de Payments

Las dependencias apuntan hacia el dominio; solo main.ts conoce los adaptadores



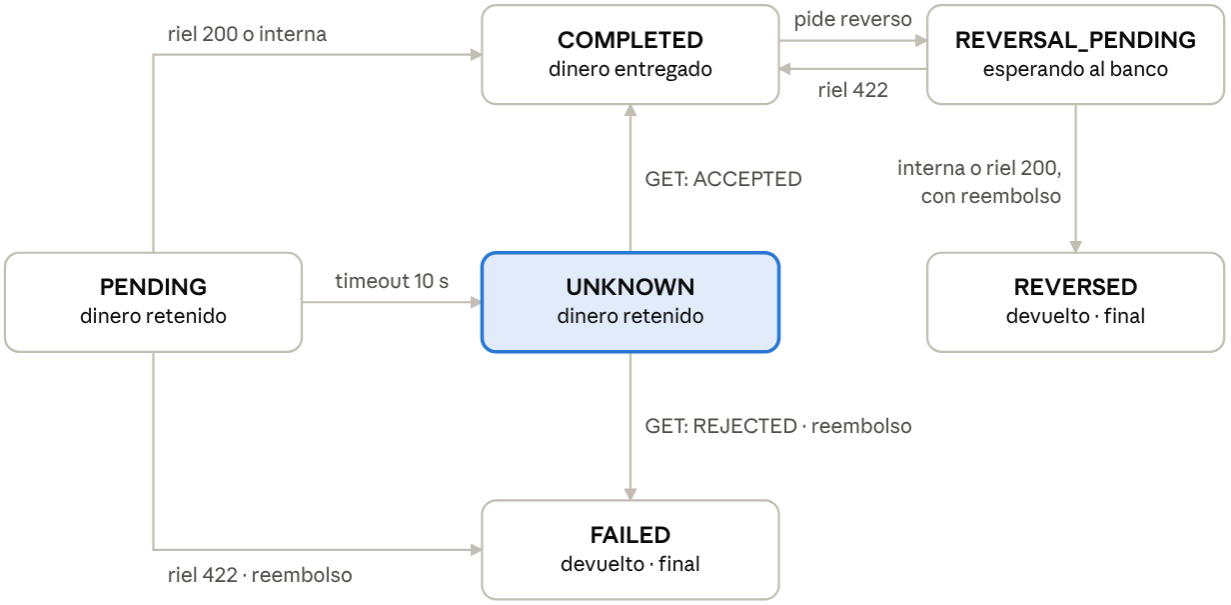
Continua: llama · discontinua: implementa un puerto del dominio

hexagonal en payments-service · puertos y adaptadores

Si preguntan dónde se ve lo hexagonal, muestra esto: `domain/` no importa pg, Fastify ni amqplib, los adaptadores implementan sus puertos y `main.ts:37-80` los conecta. El relay del outbox vive fuera del dominio: lee la tabla y publica.

Máquina de estados de la transferencia

Un timeout nunca lleva a FAILED: pasa por UNKNOWN y se reconcilia con el banco



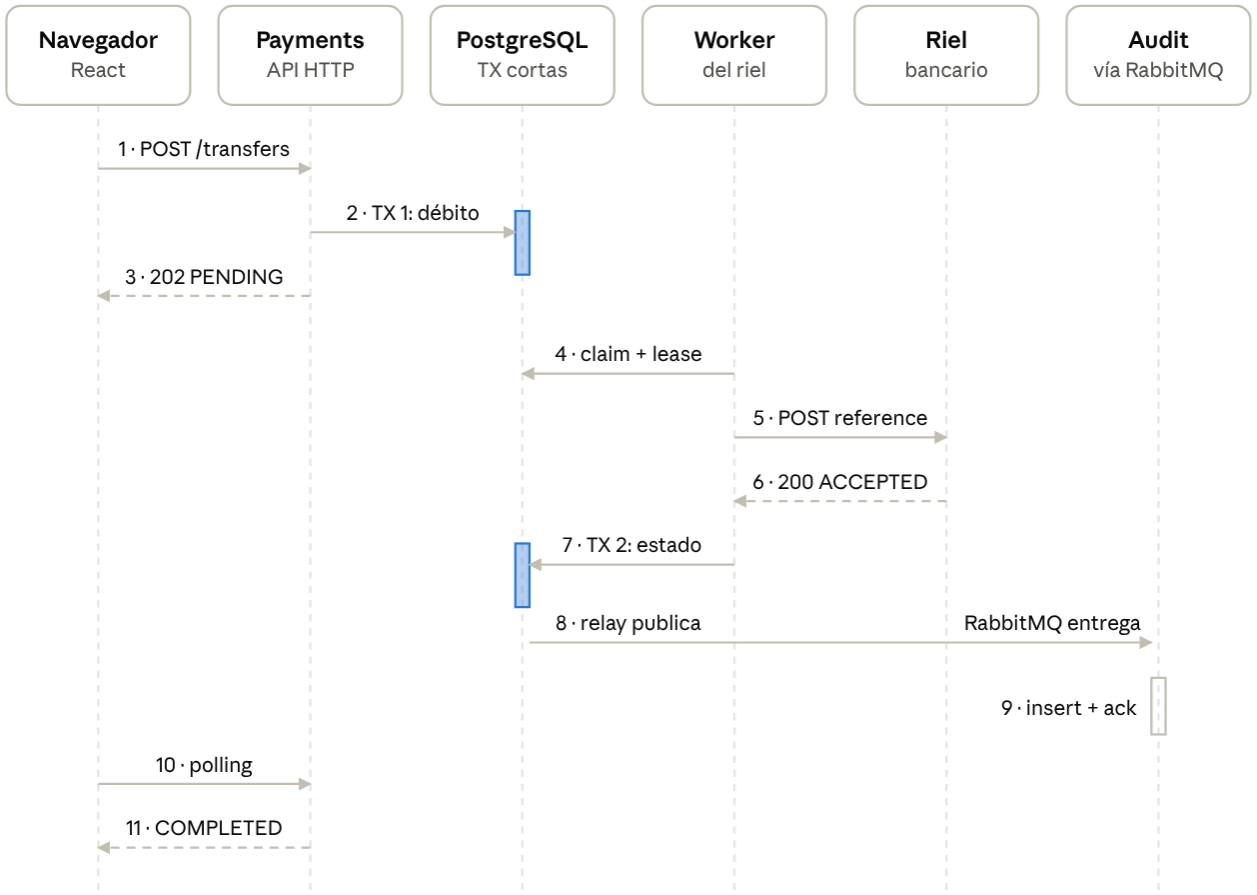
Sin cambio de estado
PENDING ante 503 o sin conexión, y UNKNOWN si el GET falla: siguen igual y reintentan con backoff.
UNKNOWN con 404 en el GET: el banco nunca la recibió y se reenvía con la misma reference.
Desde el intento 10 queda requiresManualReview; nunca se resuelve por suposición.

máquina de estados · 6 estados, 2 finales

Toda transferencia nace PENDING; la interna pasa a COMPLETED en la misma transacción. Las transiciones válidas están en `Transfer.ts:24` y la BD las refuerza con `WHERE status = esperado`, así que dos procesos no pueden aplicar la misma transición.

Una transferencia externa de punta a punta

El dinero se mueve en dos transacciones cortas; el riel se llama fuera de ellas



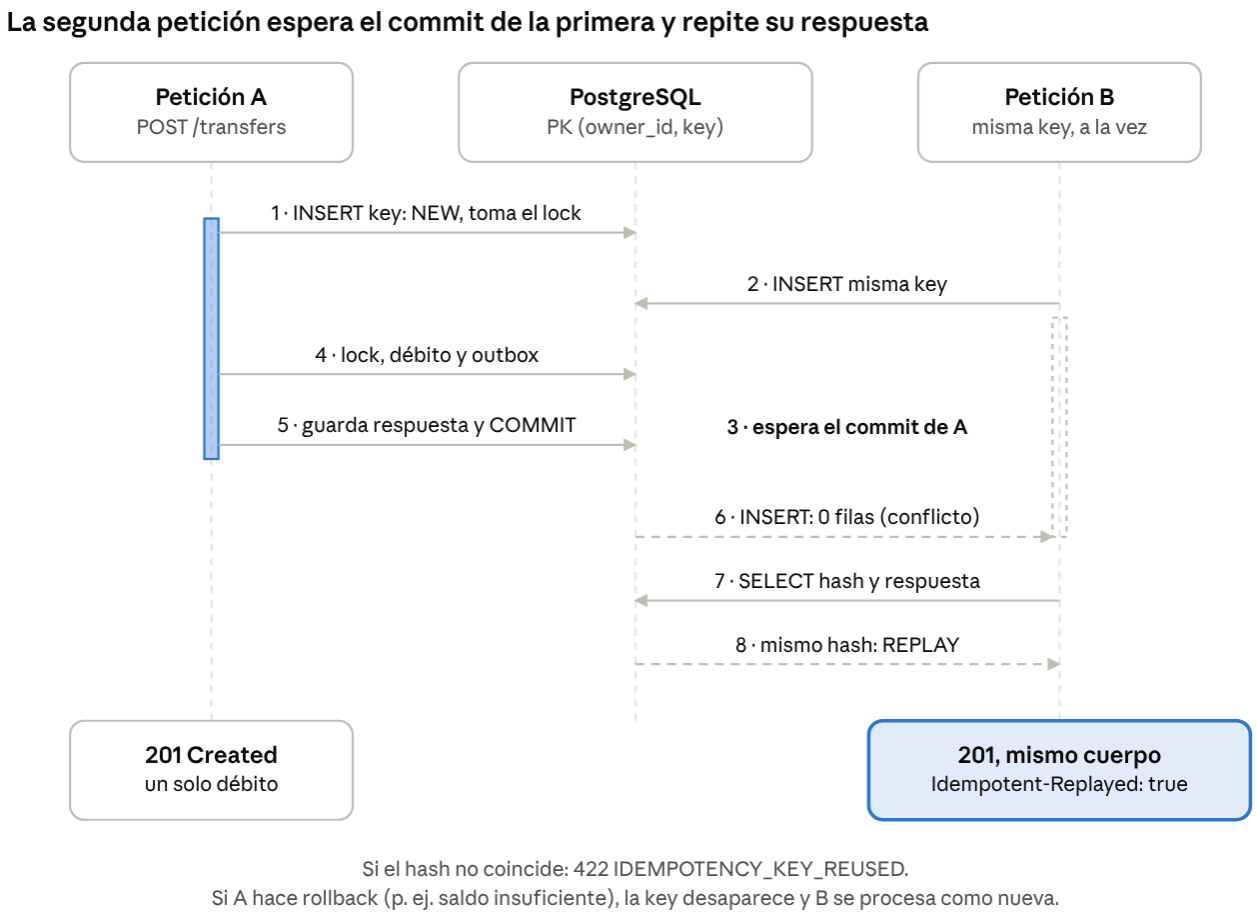
Discontinua: respuesta · barras de color: transacción en PostgreSQL

secuencia de una transferencia externa · 11 pasos

1. El navegador envía `POST /transfers` con `Idempotency-Key` (`app.ts:111`).
2. TX 1 (`CreateTransfer.ts:47`): registra la key, bloquea la cuenta, debita, guarda la transferencia PENDING y su evento en el outbox.
3. Responde 202 PENDING sin esperar al riel y despierta a los workers (`app.ts:115`).
4. Un worker reclama la transferencia con `FOR UPDATE SKIP LOCKED`, suma un intento y la alquila 30 s (`PgTransferRepository.ts:101`).
5. Fuera de cualquier transacción llama al riel con `reference` = id (`ProcessRailOperations.ts:54`). Si ya hubo un intento, antes consulta el GET.
6. El riel responde 200, o bien 422, 503 o nada en 10 s.
7. TX 2 (`ProcessRailOperations.ts:89`): COMPLETED y su evento; con 422, FAILED y reembolso en la misma transacción.
8. El relay publica el outbox con confirm y RabbitMQ lo entrega a la cola de Audit (`OutboxRelay.ts:38`).

9. Audit inserta con `ON CONFLICT (event_id) DO NOTHING` y hace ack (`rabbitmq-audit-consumer.ts:136`).
10. El detalle consulta cada 1,5 s hasta un estado estable.
11. Recibe COMPLETED; el dashboard además recibe el aviso por WebSocket.

Idempotencia con dos requests simultáneos

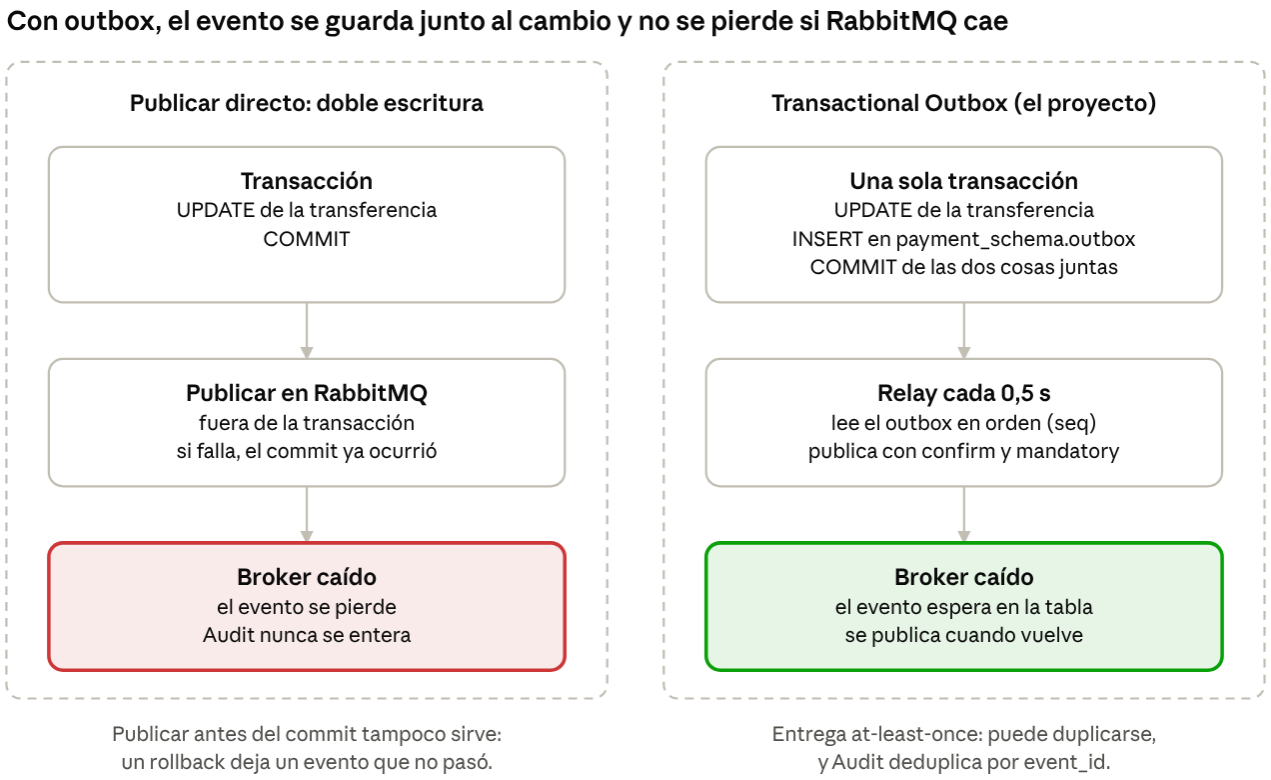


Si el hash no coincide: 422 IDEMPOTENCY_KEY_REUSED.
Si A hace rollback (p. ej. saldo insuficiente), la key desaparece y B se procesa como nueva.

idempotencia concurrente · dos peticiones, una key

Es lo que prueba IDEM-02 con 10 requests: 1 creada y 9 replays (`transfers.acceptance.test.ts:212`). No hace falta un lock distribuido ni un estado IN_PROGRESS: el índice único de PostgreSQL hace de lock (`PgIdempotencyStore.ts:14`).

Por qué Transactional Outbox



antes y después · doble escritura frente a Transactional Outbox

La prueba exige que ningún evento se pierda con el broker caído; esto lo resuelve sin bloquear la transacción principal. Pruébalo en vivo con el bloque «Broker caído» de la demo (`docker compose stop rabbitmq`).

Recorrido del código: qué abrir y qué decir

Recorre en este orden: reglas y spec, dominio, cómo se mueve el dinero, riel, eventos y bordes. En VS Code usa Cmd+P y escribe `Money.ts:16` para saltar directo a la línea.

Spec, reglas y arquitectura (raíz del repo)

Qué mostrar	Dónde	Qué decir
Reglas del agente	`CLAUDE.md` §2 a §5	Dinero, idempotencia, estados y outbox quedaron fijados antes del código
Un criterio	`docs/spec.md:142` (IDEM-02)	Cada criterio tiene ID y al menos un test con ese ID

Qué mostrar	Dónde	Qué decir
Trazabilidad	`scripts/trace.mjs:10` → `docs/traceability.md`	Falla si un criterio no tiene test o un test cita un ID inexistente
Aislamiento de esquemas	`db/init/01-schemas-and-roles.sql:6-14`	Un rol por servicio, dueño solo de su esquema

Payments: dominio y entrada (bajo `services/payments-service/src/`)

Qué mostrar	Dónde	Qué decir
Dinero en centavos	`domain/model/Money.ts:16` y `:38`	`bigint`; restar más de lo que hay lanza INSUFFICIENT_FUNDS
Máquina de estados	`domain/model/Transfer.ts:24` y `:209`	Tabla de transiciones; cualquier otra es INVALID_STATE (409)
Timeout nunca es FAILED	`domain/model/Transfer.ts:169` y `:193`	UNKNOWN y backoff; desde el intento 10, revisión manual
Puertos	`domain/ports/repositories.ts:32` y `:66`	El dominio pide interfaces; no conoce pg, Fastify ni RabbitMQ
Composition root	`main.ts:37-80`	Único archivo que conecta adaptadores con casos de uso
Entrada HTTP	`infrastructure/http/app.ts:111` y `:197`	Sin Idempotency-Key responde 400 antes de tocar nada
Errores a HTTP	`infrastructure/http/app.ts:37`	El dominio lanza códigos; el adaptador los traduce a 403, 404, 409 o 422
JWT	`infrastructure/http/app.ts:81` y `:218`	Valida firma, emisor, audiencia y vencimiento con HS256 fijo

Payments: mover dinero

Qué mostrar	Dónde	Qué decir
Crear transferencia	`application/CreateTransfer.ts:39`, `:49`, `:68`	Huella canónica, la key primero, respuesta guardada antes del commit
Mover el dinero	`application/executeTransfer.ts:24`, `:42`, `:51`	Bloqueo, débito y evento en la misma transacción
La key como lock	`infrastructure/persistence/PgIdempotencyStore.ts:14` y `:30`	El INSERT concurrente espera el commit del primero y luego reproduce
Orden de bloqueo	`infrastructure/persistence/PgAccountRepository.ts:36`	`ORDER BY id FOR UPDATE`: siempre el mismo orden, sin deadlocks
Transacción	`infrastructure/persistence/PgUnitOfWork.ts:13`	Un BEGIN por caso de uso; si algo lanza, ROLLBACK de todo
Restricciones en BD	`../migrations/001_init.sql:5`, `:37`, `:46`	CHECK de saldo, PK de idempotencia y tabla outbox
Reverso	`application/ReverseTransfer.ts:36`, `:53`, `:60`	Fila bloqueada y UPDATE condicional: una sola vez, también concurrente

Payments: riel bancario

Qué mostrar	Dónde	Qué decir
Worker	`application/ProcessRailOperations.ts:36`, `:76`, `:114`	Reclama, llama al riel fuera de la transacción y aplica el resultado en otra

Qué mostrar	Dónde	Qué decir
Claim con lease	`infrastructure/persistence/PgTransferRepository.ts:101`	`FOR UPDATE SKIP LOCKED`: 8 workers se reparten sin pisarse
UPDATE condicional	`infrastructure/persistence/PgTransferRepository.ts:77`	`WHERE status = esperado`; 0 filas = conflicto
Adaptador HTTP	`infrastructure/rail/HttpBankRail.ts:59`, `:65`, `:71`	Timeout de 10 s; no conectar = no llegó; sin respuesta = desconocido
Simulador	`services/bank-rail-simulator/src/rail.ts:89` y `:111`	Palabras en la cuenta destino fuerzan cada resultado para la demo

Eventos y Audit

Qué mostrar	Dónde	Qué decir
Relay del outbox	`payments-service/src/infrastructure/messaging/OutboxRelay.ts:43`, `:64`, `:66`	Publica en orden; si el broker cae, corta el lote y reintenta
Confirms	`payments-service/src/infrastructure/messaging/RabbitMqPublisher.ts:46` y `:49`	`persistent` y `mandatory`: solo se marca publicado tras el confirm
Consumidor	`audit-service/src/infrastructure/messaging/rabbitmq-audit-consumer.ts:92`, `:136`, `:165`	Ack manual tras el insert; mensaje inválido a la DLQ
Insert idempotente	`audit-service/src/infrastructure/persistence/pg-audit-event-repository.ts:24` y `:56`	`ON CONFLICT (event_id) DO NOTHING`; solo los participantes leen

Auth y frontend

Qué mostrar	Dónde	Qué decir
Login	`auth-service/src/application/login.ts:30`	Hash señuelo: no se sabe qué emails existen midiendo tiempos
Refresh rotado	`auth-service/src/application/refresh-session.ts:31`, `:36`, `:61`	Reusar un token rotado revoca todas las sesiones del usuario
Doble clic	`frontend/src/components/TransferForm.tsx:25`, `:30`, `:65`, `:69`	Key por intento; ref síncrona; key nueva tras éxito o tras un 4xx editado
Polling y Reversar	`frontend/src/pages/TransferDetailPage.tsx:10` y `:227`	Polling cada 1,5 s; Reversar solo para el dueño del origen
Un solo refresh	`frontend/src/api/client.ts:98` y `:130`	Varias peticiones con 401 comparten un único refresh
Tiempo real	`frontend/src/realtime/useTransferUpdates.ts:25`	Cierre 4401: renueva el token una vez y reconecta

Garantías críticas y el test que las prueba

Cada garantía tiene un mecanismo en el código y un test con el ID del criterio. Los de aceptación corren contra PostgreSQL real (Testcontainers); el consumidor de Audit y el fan-out de tiempo real, también contra RabbitMQ real.

Garantía	Mecanismo	Test (archivo:línea)
Nunca saldo negativo	`Money.subtract` lanza antes de mutar y `CHECK (balance >= 0)` en BD	payments `transfers.acceptance.test.ts:56` y `:227` (10 concurrentes, pasan 8)

Garantía	Mecanismo	Test (archivo:línea)
10 requests con la misma key = 1 débito	`INSERT … ON CONFLICT DO NOTHING` en la misma transacción; el segundo espera el lock del índice	payments `transfers.acceptance.test.ts:212` (IDEM-02: 1 creada, 9 replays)
Misma key con otro payload = 422	SHA-256 de una huella canónica (`100.5` = `100.50`)	payments `transfers.acceptance.test.ts:239` y `:203`
Reverso una sola vez, también concurrente	`FOR UPDATE` de la fila, transición validada y UPDATE condicional	payments `reversals.acceptance.test.ts:81` (REV-03)
El reverso no deja negativo al destino	`destination.debit` lanza y la transacción hace rollback	payments `reversals.acceptance.test.ts:94` (REV-04)
Timeout nunca es FAILED	Sin respuesta = UNKNOWN; el dinero sigue retenido	payments `rail.acceptance.test.ts:86` (EXT-05)
Consultar antes de reenviar	Si `attempts > 1` , primero `GET /rail/transfers/{reference}`	payments `rail.acceptance.test.ts:133` y `:144` (EXT-07)
El dinero se conserva con cualquier respuesta del riel	Reembolso en la misma transacción que el FAILED	payments `rail.acceptance.test.ts:165` (EXT-08)
Evento en la misma transacción	Insert en `payment_schema.outbox` dentro del mismo BEGIN	payments `outbox.acceptance.test.ts:47` y `:61` (rollback = sin evento)
Broker caído no bloquea ni pierde	El relay corta el lote; los eventos esperan en la tabla	payments `outbox.acceptance.test.ts:74` (AUD-02)
Publicado solo tras el confirm	Confirm channel y `mandatory`	payments `outbox.acceptance.test.ts:90` (AUD-03, con broker falso del puerto)
Consumidor idempotente	PK `event_id` y `ON CONFLICT DO NOTHING`	audit `rabbitmq-audit-consumer.test.ts:91` (AUD-04)

Garantía	Mecanismo	Test (archivo:línea)
Esquemas aislados	Un rol por esquema	payments `schema-isolation.acceptance.test.ts:14` y audit `audit-service.test.ts:130` (ARQ-01)
Refresh concurrente rota una sola vez	`FOR UPDATE` sobre el refresh token	auth `auth.acceptance.test.ts:235` (AUTH-05)
Doble clic = una petición	`useRef` síncrono además del botón deshabilitado	frontend `transfer-form.test.tsx:30` (FE-01)

Para correrlos todos: `./scripts/test-all.sh` desde la raíz (necesita Docker encendido).

Tests por módulo



README del repo · tests por módulo al 2026-10-15

El peso está donde está el riesgo: idempotencia, concurrencia, riel y outbox viven en Payments, y sus pruebas de aceptación corren contra PostgreSQL real.

Demo en vivo: guion y comandos

Haz la demo en local (http://localhost:8080), porque ahí puedes apagar RabbitMQ y cambiar el modo del riel; https://98-81-60-52.sslip.io queda de respaldo. Los comandos de abajo los probé contra tu stack local y funcionan tal cual en zsh.

En la interfaz (alice@demo.co / Demo1234!)

1. Inicia sesión: saldos de ACC-001 y movimientos con el indicador «En vivo». Abre una ventana privada con bob@demo.co para que vean llegar los avisos sin recargar.
2. Transferencia interna de 100000.00 a ACC-002: queda COMPLETED al instante y el timeline muestra CREATED y COMPLETED.
3. Externa a `EXT-OK-1`: responde 202 PENDING y en un segundo pasa a COMPLETED.
4. Externa a `EXT-REJECT-1`: queda FAILED y el saldo vuelve en la misma transacción.
5. Externa a `EXT-TIMEOUT-1`: a los 10 s pasa a UNKNOWN. El simulador sí la procesó y responde a los 15 s; en el siguiente intento el worker consulta el GET y la deja COMPLETED sin reenviar. Si el GET cae en su fallo aleatorio (15 %), verás un reintento más.
6. Reversa la transferencia interna del paso 2: REVERSED y saldos restaurados. Con bob abierto, muestra que bob no tiene botón Reversar.

Las palabras OK, REJECT, DOWN, TIMEOUT y LOST en la cuenta destino fuerzan el resultado del riel; deben ir como palabra completa (`EXT-OK-1` sí, `EXTOK` no).

Idempotencia con curl

```
TOKEN=$(curl -s localhost:8080/api/auth/login -H 'content-type:
application/json' \
  -d '{"email":"alice@demo.co","password":"Demo1234!"}' | jq -r .accessToken)
t() { curl -si localhost:8080/api/transfers -H "authorization: Bearer $TOKEN"
-H 'content-type: application/json' "$@"; }
KEY=$(uuidgen)
BODY='{"type":"INTERNAL","fromAccount":"ACC-001","toAccount":"ACC-
002","amount":"1000.00"}'

t -H "idempotency-key: $KEY" -d "$BODY" | head -1                          #
201 Created
t -H "idempotency-key: $KEY" -d "$BODY" | grep -i 'HTTP/\|replayed'        #
201 + idempotent-replayed: true
t -H "idempotency-key: $KEY" -d "${BODY/1000.00/2000.00}" | tail -1        #
422 IDEMPOTENCY_KEY_REUSED
t -d "$BODY" | tail -1                                                      #
400 IDEMPOTENCY_KEY_REQUIRED

# 10 en paralelo con la misma key: diez 201, una sola transferencia y un solo
débito
K2=$(uuidgen); ( for i in $(seq 10); do t -o /dev/null -w '%{http_code} ' -H
"idempotency-key: $K2" -d "$BODY" & done; wait ); echo
```

Broker caído: no bloquea ni pierde eventos

```
docker compose stop rabbitmq      # haz una transferencia en la UI: responde
igual
docker compose exec -T postgres psql -U postgres -d payments_portal \
  -c "select event_type, published_at from payment_schema.outbox order by seq
desc limit 4;"
docker compose start rabbitmq     # en segundos se llena published_at y
aparece el timeline
```

Riel caído y recuperación

```
curl -s -X PUT localhost:8081/rail/admin/mode -H 'content-type:
application/json' -d '{"mode":"UNAVAILABLE"}'
# Externa a EXT-123 en la UI: sigue PENDING y sube «Intentos con el banco»
(backoff 1, 2, 4, 8 s…)
curl -s -X PUT localhost:8081/rail/admin/mode -H 'content-type:
application/json' -d '{"mode":"ACCEPT"}'
# Siguiente intento: GET da 404 (nunca la procesó), reenvía con la misma
reference y queda COMPLETED
curl -s -X PUT localhost:8081/rail/admin/mode -H 'content-type:
application/json' -d '{"mode":"RANDOM"}'
```

No dejes el riel caído más de unos 20 s: el backoff crece hasta 60 s y la demo se vuelve lenta. La consola de RabbitMQ está en http://localhost:15672 (guest / guest), con la cola `audit.transfer-events` y su DLQ.

Preguntas probables y respuestas cortas

Responde en dos frases y abre el archivo si piden detalle. Las incómodas van al final: reconoce el límite y di qué harías.

SDD y uso de la IA

- **¿Qué te dio SDD en 3 horas?** La spec fue el contrato con el agente: cada criterio tiene ID, test y código. Cuando el agente se equivocaba, la spec decidía (la fórmula de EXT-08 se corrigió primero en la spec).
- **¿Qué hizo el agente y qué hiciste tú?** El agente escribió la mayor parte del código y los tests, con cuatro subagentes en paralelo. Yo decidí stack y alcance, revisé cada diff, corrí las pruebas y aprobé cada commit.
- **¿Cómo controlaste a los subagentes?** Brief cerrado: `CLAUDE.md`, criterios de la spec, `openapi.yaml` y el contrato del evento. Tenían prohibido tocar fuera de su carpeta o hacer commits.

- **¿Cómo sabes que el código del agente está bien?** Tests con el ID del criterio, `trace.mjs` en 59/59, PostgreSQL real y el flujo completo en compose. Así salieron los 9 errores del README.

Arquitectura

- **¿Dónde se ve lo hexagonal?** `domain/` y `application/` no importan pg, Fastify, amqplib ni `process.env`; los puertos están en `domain/ports/` y `main.ts` arma todo. Por eso el worker del riel se prueba con un riel falso.
- **¿Por qué Node.js?** Un solo lenguaje en backend y frontend, TypeScript estricto y arranque rápido. Fastify solo vive en `infrastructure/http`.
- **¿Por qué RabbitMQ y no Kafka?** Para eventos con ack por mensaje, DLQ y fan-out por cola, RabbitMQ es directo. Kafka lo usaría para reprocesar el historial completo o con mucho volumen.
- **¿Por qué un rol por esquema?** Para que el aislamiento lo garantice PostgreSQL y no una convención: leer otro esquema falla con permiso denegado (ARQ-01).
- **¿Cómo sabe Audit quién puede ver un evento sin leer pagos?** Cada evento lleva `participantIds` y Audit filtra por ellos (`pg-audit-event-repository.ts:56`).
- **¿Por qué SQL explícito y no un ORM?** La consistencia depende de `FOR UPDATE`, `SKIP LOCKED`, `ON CONFLICT` y UPDATE condicionales; un ORM los esconde.
- **¿Cómo escala Payments?** No guarda estado en memoria: idempotencia en BD, workers con `SKIP LOCKED` y lease, y una cola exclusiva por instancia para tiempo real (RT-03). El límite es PostgreSQL.

Dinero y concurrencia

- **¿Por qué `bigint`?** `0.1 + 0.2` no da `0.3` con `number`. En centavos la aritmética es exacta, y `pg` entrega NUMERIC como string sin pasar por `number`.
- **¿Cómo evitas el doble gasto con keys distintas?** `SELECT … FOR UPDATE` serializa sobre la cuenta origen y el segundo ve el saldo ya debitado. Con 10 × 60.000 sobre 500.000 pasan exactamente 8 (TR-04).
- **¿Y los deadlocks?** Todas las transacciones bloquean cuentas en orden de id (`PgAccountRepository.ts:36`), así A→B y B→A piden los locks en el mismo orden.
- **¿Por qué no SERIALIZABLE?** READ COMMITTED con bloqueo explícito de las filas involucradas basta y evita reintentos por fallos de serialización.
- **¿Y si el proceso muere a mitad?** Key, débito, transferencia y evento se confirman juntos o no existen. El cliente reintenta con la misma key.

Idempotencia

- **¿Por qué la key en la misma transacción?** En dos pasos, una caída entre ambos deja key sin operación u operación sin key. Juntas, el índice único hace de lock y no hace falta un estado IN_PROGRESS.
- **¿Dos requests simultáneos con la misma key?** El segundo INSERT espera el commit del primero, lee la respuesta guardada y la reproduce con `Idempotent-Replayed: true`.
- **¿Se guardan los errores?** No, solo respuestas exitosas. Un 422 hace rollback también de la key y el reintento se reevalúa; el frontend genera key nueva si el usuario edita.
- **¿Por qué un hash del payload?** Para rechazar la misma key con otro cuerpo (422). La huella es canónica: `100.5` y `100.50` son el mismo request (`CreateTransfer.ts:39`).
- **¿Las keys vencen?** Todavía no; falta un TTL con limpieza periódica (está en Pendientes).

Riel bancario

- **¿Por qué un timeout no es FAILED?** El banco pudo procesarla. Marcarla FAILED y reembolsar sacaría el dinero dos veces; por eso queda UNKNOWN y retenida.
- **¿Qué es consultar antes de reenviar?** Si un intento anterior pudo llegar al banco, primero `GET /rail/transfers/{reference}`. Solo se reenvía con un 404; si el GET falla, no se reenvía a ciegas.
- **¿Por qué la reference es el id de la transferencia?** Es estable en todos los reintentos, así el banco deduplica (RAIL-02).
- **¿Por qué llamar al riel fuera de la transacción?** Para no retener locks ni conexiones hasta 10 s. El resultado se aplica en otra transacción que verifica que el estado no cambió (`ProcessRailOperations.ts:92`).
- **¿Y si el worker muere después de llamar al riel?** El lease vence a los 30 s, otro worker la reclama, consulta el GET y aplica la verdad. No hay doble débito ni doble reembolso.
- **¿Y si nunca se resuelve?** Desde el intento 10 queda `requiresManualReview` y sigue reintentando con backoff de hasta 60 s. Nunca se resuelve por suposición.

Eventos y auditoría

- **¿Por qué outbox y no publicar directo?** Es la doble escritura: publicar después del commit puede fallar y perder el evento; antes, puede publicar algo que luego hace rollback.
- **¿Puede llegar un evento dos veces?** Sí, la entrega es at-least-once: si falla marcar `published_at` tras el confirm, se republica. Audit deduplica por `event_id`.

- **¿Se respeta el orden?** El relay lee por `seq` y corta el lote al primer fallo; Audit procesa en serie. Con varios relays podría desordenarse, por eso el historial se ordena por `occurredAt`.
- **¿Por qué polling y no CDC?** Es simple y 500 ms de latencia bastan aquí. A más escala usaría CDC (Debezium) sobre la tabla outbox.
- **¿Qué va a la DLQ?** Mensajes que no son JSON, no cumplen el contrato o PostgreSQL rechaza por datos. Un fallo transitorio se reencola tras 1 s.

Seguridad y frontend

- **¿Riesgo de HS256 compartido?** Cualquier servicio con el secreto puede emitir tokens. Lo siguiente es RS256 con JWKS: solo Auth firma y los demás verifican con la llave pública.
- **¿Por qué el JWT va en la query del WebSocket?** El navegador no permite headers en un WebSocket. nginx no registra esa ruta, el token dura 15 min y se cierra con 4401 al vencer.
- **¿Por qué WebSocket si pedían polling o SSE?** El detalle sí usa polling cada 1,5 s (FE-02). El WebSocket solo avisa qué cambió en el dashboard; SSE habría bastado y tampoco admite headers.
- **¿Cómo evitas filtrar información?** Hash señuelo en el login, 404 (no 403) para transferencias ajenas y lista vacía en Audit para quien no participa.
- **¿Dónde viven los tokens en el navegador?** En `sessionStorage`, que se borra al cerrar la pestaña. Ante XSS serían legibles; el siguiente paso es el refresh token en cookie `httpOnly`.

Las incómodas

- **«Esto lo escribió la IA, ¿lo entiendes?»** Sí, y lo muestras abriendo el archivo. Tu trabajo fue fijar las reglas, revisar cada diff y detectar los errores del agente, que están documentados.
- **«¿Qué no alcanzaste?»** Lo de Pendientes del README: TTL de keys, vista de revisión manual, observabilidad, JWT asimétrico, e2e automatizado y build en CI.
- **«¿Dónde está el test de deadlock?»** No hay uno dedicado a transferencias cruzadas concurrentes; la garantía viene del orden de bloqueo. Es el siguiente test que agregarías.
- **«¿Probaste el relay con RabbitMQ real?»** El relay se prueba con un broker falso que implementa el puerto. El consumidor de Audit y el fan-out de tiempo real sí corren contra RabbitMQ real, y el flujo completo se ve en compose.

- **«El historial tiene LIMIT 100 sin paginación.»** Es una limitación conocida (`PgTransferRepository.ts:95`). La solución es paginar por cursor sobre `(created_at, id)`.

Cambios en vivo ensayados

En el cambio en vivo es donde más se evalúa SDD: narra cada paso en voz alta. El cambio A lo ensayé completo en una copia aparte del servicio; typecheck, test unitario y test de aceptación pasan.

Receta para cualquier cambio

1. Criterio nuevo en `docs/spec.md` con el siguiente ID libre (p. ej. TR-10), en Given/When/Then.
2. Si cambia el contrato, `docs/openapi.yaml`: campo nuevo o código de error.
3. Test con el ID en el nombre; córrelo y muéstralo en rojo.
4. Código mínimo de adentro hacia afuera: dominio, caso de uso, adaptador.
5. Tests en verde y `npm run typecheck`.
6. `node scripts/trace.mjs` desde la raíz: debe seguir sin criterios huérfanos.
7. Commit pequeño con rutas explícitas: `feat(payments): monto máximo por transferencia (TR-10)`.

```
cd services/payments-service
npx vitest run test/unit/transfer.test.ts           # unitarios, sin Docker,
menos de 1 s
npm run typecheck
npx vitest run test/acceptance/transfers.acceptance.test.ts -t "TR-10"   #
PostgreSQL real, unos 5 s
cd ../.. && node scripts/trace.mjs
```

Si piden algo que no ensayaste, busca el criterio más parecido que ya tenga test y úsalo de plantilla.

A. Monto máximo por transferencia (ensayado, unos 10 minutos)

Spec, después de TR-09:

```
**TR-10 Monto máximo por transferencia**
- When alice transfiere más de "5000000.00" en una sola transferencia
- Then recibe 422 `AMOUNT_LIMIT_EXCEEDED` y no se crea transferencia ni
cambia ningún saldo.
```

Código, bajo `services/payments-service/src/`:

1. `domain/errors.ts:14`: agrega `| 'AMOUNT_LIMIT_EXCEEDED'` al tipo `DomainErrorCode`.
2. `domain/model/Transfer.ts`: la constante arriba y la regla en `create()`, justo después del chequeo de INVALID_AMOUNT (línea 101).
3. Corre `npm run typecheck`: falla en `infrastructure/http/app.ts:37`. Muéstralo: el compilador te obliga a decidir el status HTTP. Agrega `AMOUNT_LIMIT_EXCEEDED: 422`.
4. `docs/openapi.yaml:487`: agrega el código al enum de errores.

```
// domain/model/Transfer.ts, debajo de los imports
const MAX_TRANSFER_AMOUNT = Money.fromString('5000000.00'); // TR-10

// dentro de Transfer.create(), después del chequeo de INVALID_AMOUNT
if (MAX_TRANSFER_AMOUNT.isLessThan(input.amount)) {
  throw new DomainError('AMOUNT_LIMIT_EXCEEDED', 'El monto supera el máximo
por transferencia (5.000.000,00 COP)');
}
```

Tests: el unitario va en `test/unit/transfer.test.ts`, debajo del de TR-02 (línea 23); el de aceptación, en `test/acceptance/transfers.acceptance.test.ts`, dentro de `describe('TR: creación')`.

```
it('[TR-10] el monto no puede superar 5.000.000,00 por transferencia', () =>
{
  expect(() => newTransfer('INTERNAL', '5000000.00')).not.toThrow();
  expect(() => newTransfer('INTERNAL', '5000000.01')).toThrowError(
    expect.objectContaining({ code: 'AMOUNT_LIMIT_EXCEEDED' }),
  );
});

it('[TR-10] por encima del máximo se rechaza con 422 y no mueve dinero',
async () => {
  const response = await h.createTransfer(USERS.alice,
internal('5000000.01'));

  expect(response.statusCode).toBe(422);
  expect(response.json().code).toBe('AMOUNT_LIMIT_EXCEEDED');
  expect(await countRows(db.pool, 'transfers')).toBe(0);
  expect(await balanceOf(db.pool, 'ACC-001')).toBe('1000000.00');
});
```

Qué decir mientras lo haces:

- La regla vive en el dominio, así también aplica al pago de Payment Links, que reutiliza `executeTransfer`.
- Es 422 porque es una regla de negocio (`CLAUDE.md` §7).
- El frontend no necesita cambios: muestra el mensaje del backend para códigos que no conoce (`frontend/src/api/client.ts:214`).
- Si lo piden configurable, no leas `process.env` en el dominio: pásalo desde `main.ts` al caso de uso.

B. Descripción opcional en la transferencia (unos 20 minutos)

1. Spec TR-11 y `openapi.yaml`: campo `description` en el request y en `Transfer`.
2. `migrations/004_transfer_description.sql` con `ALTER TABLE payment_schema.transfers ADD COLUMN description text CHECK (char_length(description) <= 140);`. Es nullable, no rompe filas existentes y `migrate.ts` la aplica al arrancar.
3. `domain/model/Transfer.ts`: `description: string | null` en `TransferProps`, en `create()` y un getter.
4. `infrastructure/persistence/PgTransferRepository.ts`: `COLUMNS` (línea 24), el INSERT con `$16` (línea 49) y `toTransfer` (línea 27).
5. `application/CreateTransfer.ts`: el comando y la huella de idempotencia (línea 39). Si no la incluyes, la misma key con otra descripción se reproduce en vez de dar 422.
6. `application/executeTransfer.ts` y `application/paymentLinks.ts:114`, que también lo llama.
7. `application/views.ts:32` y el zod de `infrastructure/http/app.ts:49`: `description: z.string().trim().max(140).optional()`.
8. Frontend: `api/types.ts`, el campo en `TransferForm.tsx` y una fila en `TransferDetailPage.tsx`.

Tests de aceptación: crear con descripción y verla en el GET; misma key con otra descripción, 422.

C. Límite diario por cuenta (unos 25 minutos)

1. Spec con la zona horaria explícita: el día es el de Bogotá (UTC−5 sin horario de verano, empieza a las 05:00 UTC).
2. Puerto nuevo en `domain/ports/repositories.ts:13`: `sumOutgoingSince(accountId, since)`.

3. Adaptador: `SELECT COALESCE(SUM(amount), 0)::text FROM payment_schema.transfers WHERE from_account = $1 AND created_at >= $2 AND status NOT IN ('FAILED', 'REVERSED')`.
4. Chequeo en `application/executeTransfer.ts` justo después de `lockByIds` (línea 24).

Lo que debes decir: el lock de la cuenta origen serializa las transferencias concurrentes de esa cuenta, así la suma no tiene carreras. Pruébalo con dos requests concurrentes que juntas superan el límite: pasa solo una.

D. Parámetros del riel por variable de entorno (unos 5 minutos)

- `main.ts:44-50`: `maxAttempts: Number(process.env.RAIL_MAX_ATTEMPTS ?? 10)`, y lo mismo para `baseDelayMs`.
- Agrégalas en `docker-compose.yml:75-81` y en `.env.example`.
- Solo `main.ts` lee `process.env`; el dominio y los casos de uso reciben valores (`CLAUDE.md` §1).

Errores del agente: cómo contarlos

Te pedirán al menos uno. Cuenta dos con esta estructura: qué pasó, cómo se detectó, qué se cambió y qué aprendiste. Los mejores para contar son el 2 y el 8.

#	Error	Cómo se detectó	Arreglo
1	Healthchecks con `localhost`: en alpine resuelve a `::1` y el `wget` de busybox no reintenta por IPv4	Probando la imagen de Audit	`127.0.0.1` en los healthchecks
2	Head-of-line en el worker del riel: un timeout de 10 s frenaba a las demás	Prueba e2e en compose: `EXT-DOWN-1` seguía con `attempts: 0`	8 workers que reclaman una transferencia cada uno, seguro por `SKIP LOCKED` y lease
3	Test que simulaba conexión rechazada con el puerto 1	Parecía un bug del adaptador; `fetch` bloquea ese puerto con otro error	Puerto recién liberado: ECONNREFUSED real

#	Error	Cómo se detectó	Arreglo
4	La fórmula de conservación de EXT-08 excluía las externas COMPLETED	Revisión de la spec	Corregida antes de `spec: initial`
5	`regex().refine()` de zod en el simulador: el refine corre aunque falle el regex y respondía 500	Test del simulador	Una sola función de validación y su test
6	El reconnect de Audit esperaba el `close()` de un socket medio abierto y podía colgarse	Revisión del código	Programa el reintento primero y no espera el `close`
7	Supuso que el riel lo entregaban los evaluadores	Correo de aclaración	Cambio de spec y simulador propio
8	El test de PL-02 pasaba sin probar nada: corría sin sesión iniciada	Revisión del test	Variante con sesión, que sí falla si se envía el token
9	El botón Reversar aparecía también para el receptor	Revisión de las pantallas de Payment Links	Solo lo ve el dueño del origen (REV-01 del frontend)

El 2 en 30 segundos. La primera versión del worker tomaba un lote y no reclamaba más trabajo hasta terminarlo. En la prueba con compose, una transferencia a `EXT-DOWN-1` seguía en `attempts: 0` porque otra esperaba un timeout de 10 s. Se cambió por 8 workers de una transferencia cada uno, seguro gracias a `FOR UPDATE SKIP LOCKED` con lease. Aprendizaje: los tests unitarios no lo mostraban, hizo falta la prueba end-to-end.

El 8 en 30 segundos. El test decía «la consulta pública no envía el token», pero corría sin sesión, así que pasaba aunque el código enviara el token. Se agregó la variante con sesión. Aprendizaje: ante cada test del agente, pregúntate qué cambio del código lo haría fallar.

Checklist antes de entrar

La prueba pide tener el proyecto corriendo en tu máquina: deja todo abierto 15 minutos antes.

- [x] ~~Stack local arriba: `docker compose ps` con todo en `healthy`.~~
- [x] ~~Datos limpios (opcional): `docker compose down -v && docker compose up -d --build` deja los saldos en 1.000.000 / 500.000 / 0. Solo borra la base local.~~

- [x] ~~Tests en verde con `./scripts/test-all.sh`. Tarda unos minutos: córrelo antes, no durante la sesión.~~
- [x] ~~VS Code en la raíz con estas pestañas: `CLAUDE.md`, `docs/spec.md`, `Transfer.ts`, `CreateTransfer.ts`, `PgIdempotencyStore.ts` y `ProcessRailOperations.ts`.~~
- [x] ~~Navegador con alice en http://localhost:8080, bob en una ventana privada y la consola de RabbitMQ.~~
- [x] ~~Terminal con el bloque de curl de la demo ya pegado.~~
- [x] ~~Respaldo: https://98-81-60-52.sslip.io responde (si apagaste la instancia, usa solo local).~~
- [x] ~~Ensaya el cambio A una vez en una rama local y bórrala: `git switch -c ensayo`, cambios y commit, `git switch main`, `git branch -D ensayo`. No la subas.~~
- [x] ~~Lee en voz alta el pitch y las respuestas de «Las incómodas».~~

Durante la sesión, si algo falla en vivo, di qué esperabas ver y lee el error antes de tocar código. Si no sabes algo, dilo y muestra cómo lo averiguarías: abriendo el test que tiene el ID del criterio.